

AUTONOMOUS ML RESEARCH / TECHJAM 2026

SciOdyssey: Long-Horizon Autonomous ML Research Agent

Autonomous by default. [Supervisable by design.](#)

ROBUST

SUPERVISABLE

BROAD SEARCH

The run, in five moves

A single route from the problem to the proof, then the product experience.

01



PROBLEM

Why current agents fail

Set the task and its failure modes.

02



SYSTEM

A system for sustained research

Keep evidence while resetting the scientist.

03



EVALUATION

From claim to evidence

Measure what survives the run.

04



INSIGHTS

What the run taught us

Depth, breadth, recovery, and review.

Automating ML experimentation

Given a dataset and a scoring metric, an agent must build a model and improve it through repeated experiments.

01	02	03	04	05
Read the problem	Inspect data	Engineer features	Train + tune	Evaluate
What should the model predict?	What patterns does the data contain?	Turn raw data into useful model inputs	Fit a model and adjust its settings	Check performance on validation data
REFLECT + REVISE				
The agent uses the results to choose what to change, revises the code, and runs the next experiment.				

THE AGENT'S JOB

Write the code and run the full loop autonomously.

This includes building the pipeline and deciding which experiments to try next.

THE CHALLENGE GOAL

Reproduce, then improve on the organizer's baseline.

Develop with training data and public validation. The organizer scores the final submission on a hidden test set.

Why current agents fail

01 WORKFLOW



Observe → Code → **Error** → **Recover**

Coding
Harness

Runtime
Tools

Acquire missing tools at runtime.

- ✓ Resume after unexpected errors.
- ✓ Extend with Skills & MCP.

02 SINGLE AGENT



last frame — next move

history-shaped

single path

Fresh reasoning. Audited evidence.

- ✓ Reopen search.
- ✓ Check the science.

03 TREE SEARCH



READ ONCE



BRANCH EDITS

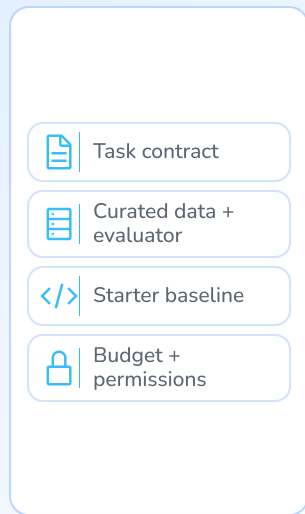
Persist the read. Vary the hypothesis.

Reuse context across experiments.

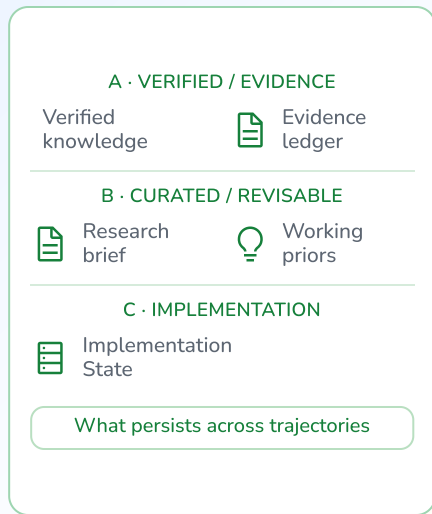
- ✓ Raise task granularity.
- ✓ Parallelize across branches.

A system for sustained research

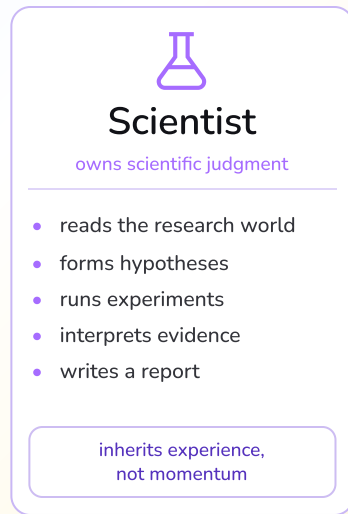
Environment



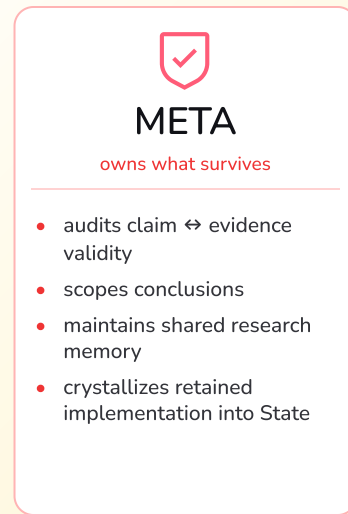
Research World



Fresh Scientist



META Review



Selective persistence / trajectory handoff

Coding-agent harness + runtime



Principles of autonomous science

Practical disciplines that make autonomous ML research grounded, robust, and cost-effective.

01 DATA BOUNDARY

EXPLORE.md

Ground the data, skip the ritual

Explore only what affects the model

Skip useless data dumps: Don't print giant tables that clutter context without helping decisions.

Check leaks before training: Verify time splits and feature cutoffs to avoid fake high scores.

03 RESOURCE CONTROL

OPTIMIZE.md

Test ideas cheap and early

Screen fast on small runs, protect full budget

Use gates to fail fast: Quickly drop weak ideas on small tests; never waste full training to rescue them.

Keep small tests realistic: Small runs must keep true data grouping (like user groups), or scores mislead.

02 EXECUTION FOUNDATION

ENGINEERING.md

Engineering before experiment

Scripts, tooling & limits make the agent robust

Build solid scripts first: Write modular entrypoints, fast diagnostics, and self-healing tools before modeling.

Estimate compute & runtime: Record CPU/GPU, memory, and task duration in `ENGINEERING.md` to prevent OOM and timeouts.

04 SELF-EVOLUTION

INTUITION.md

Learn like a human researcher

Self-evolution through high-level memory

Clear the execution noise: Wipe verbose terminal logs on resets so the agent stays fresh without context bloat.

Pass down high-level intuition: Save key rules and lessons in `INTUITION.md` to inherit experience, not momentum.

We are not tokenmaxxing

PROJECT-LEVEL RESOURCE ACCOUNTING

~\$10

TOTAL PROJECT COST

Subscription-equivalent usage from first line of code to final result.

coding · debugging · agent loops · all 11 experiments

MODEL

One GPT Plus + One Gemini Pro for 7 days

Two consumer subscriptions covered the entire autonomous research loop.

HARDWARE

1 × MacBook M2

All development, feature engineering, and validation ran on one consumer laptop.

COMPUTE

0 GPU-hours

100% CPU NumPy training and inference; zero cloud GPU clusters required.

Performance came from the autonomous research loop, not an expensive compute budget.

Model diversity breaks search plateaus

Alternating GPT and Gemini combines depth and breadth within one shared research world.

01 · DEPTH

GPT 5.6

Deepen & optimize proven paths

Disciplined step-by-step exploitation and hyperparameter refinement.

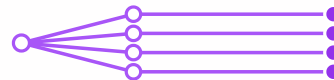


02 · BREADTH

Gemini 3.7

Open new architectural mechanisms

Broad hypothesis exploration, mechanism pivots, and candidate screening.



RETAINED 4-CYCLE ROTATION SEQUENCE

CYCLE 01

GPT 5.6

FM Baseline



CYCLE 02

Gemini 3.7

Target DIN



CYCLE 03

GPT 5.6

Feature Eng



CYCLE 04

Gemini 3.7

Ensemble

🔄 Shared Research Memory · Maintained by META across all resets

PUBLIC-VALIDATION PRIMARY

Gemini-only

0.6052

Cycle 2 plateau

GPT + Gemini

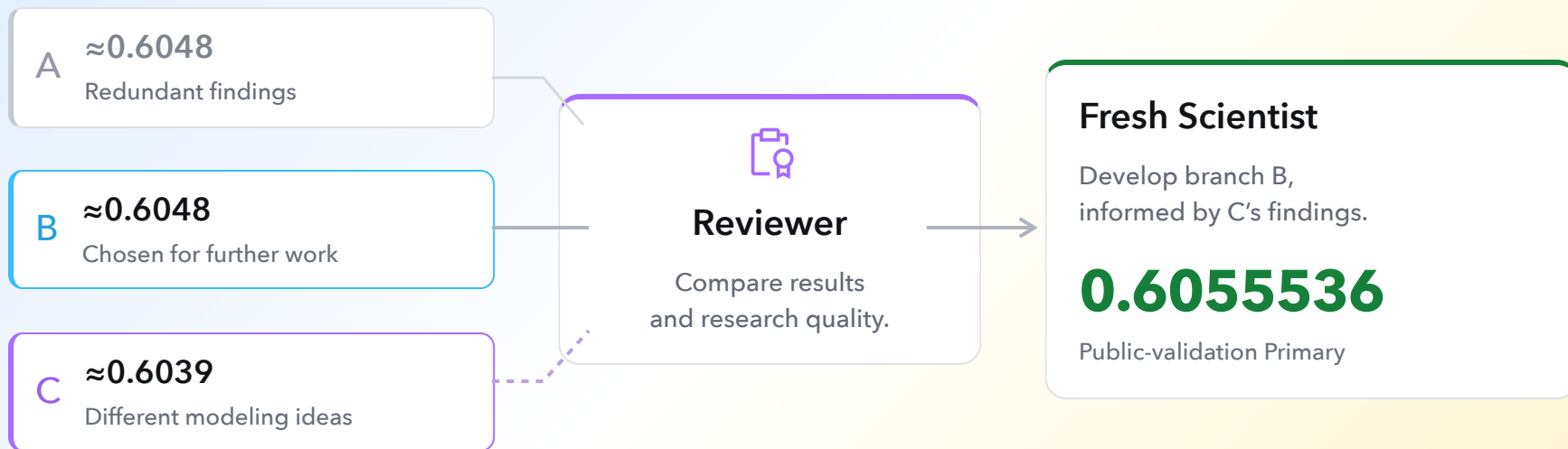
+0.0007

0.6059

Retained peak submission

Explore in parallel. Build on the findings.

One branch continues. Another contributes a useful perspective.



A lower-scoring branch can still contribute an idea.

The new model used pairwise ranking, higher rank and rank-normalized ensembling.

A clean repo. A continuous agent.

FIELD NOTE / 01

Engineering practices to apply next: isolate parallel changes and make unattended work reviewable.

01 MANY AGENTS



Give every agent a branch.

main

clean trunk

● agent A / branch

● agent B / branch

● agent C / branch

review

→ merge

Let agents work independently. Conflicts stay local, and the main branch stays clean.

02 NO QUOTA / NO LAPTOP



Leave the next move in GitHub.

01

Issue /
prompt

inject the task



02

GitHub
Action

run the agent



03

Commit

review the change

Connect the agent to GitHub Actions. It can code and commit while you are offline; review before merging.

Autonomous by default.
Supervisable by design.



Research is an **open-world** task.
We need an **open-world** coding agent.

01 OPEN ACTION SPACE



Shell



Files



Browser



Git

Use what exists

02 SELF-RECOVERY



Read errors



Inspect docs



Inspect
source



Install
packages

Fix what breaks

03 RUNTIME EXTENSION



MCP



Skills



CLI



Packages

Add what's missing

Five people. One autonomous research loop.

OVERLAPPING RESPONSIBILITIES / SHARED WORLD



Chen Zhu

TEAM LEAD / SYSTEM

Direction · architecture ·
integration



Zhou Ziyu

MODELING / EXPERIMENTS

Features · ensembles ·
evidence review



Shilin Xu

DATA / METRICS

Data workflow · contracts
· alignment



GE GAO

RUNTIME / RELIABILITY

Execution · testing ·
integration support



Jiran Li

EVIDENCE /
REPRODUCIBILITY

Telemetry · documentation
· packaging